SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

Name: Charuthya P

Registration number: 192421148

COURSE NAME: Software Testing

COURSE CODE: CSA37

1. Reverse engineer an ATM withdrawal module and reconstruct its black-box test strategy.

Answer

Introduction:
The ATM withdrawal module allows a customer to withdraw money from an account. Since only input-output behaviour is available, reverse engineering means observing the responses and inferring the rules used by the system.

Functional components:

1. Card/PIN validation – checks whether the card and PIN are valid.

2. Account validation – verifies the account and its status.

3. Withdrawal amount validation – checks whether the amount is positive and within the ATM limit.

4. Balance validation – ensures the account has enough funds.

5. Cash availability – confirms the ATM has enough cash.

6. Transaction processing – deducts the amount and dispenses cash.

7. Receipt/error handling – displays success or an appropriate error.

Equivalence partitions:

• Valid amount: positive amount within the permitted withdrawal limit and valid denomination.

• Invalid amount: zero, negative, non-numeric, or unsupported denomination.

• Balance classes: amount below available balance, exactly equal to balance, and above balance.

• Limit classes: below daily limit, exactly at the limit, and above the limit.

• Card/PIN classes: valid card and PIN; invalid, blocked, or expired card; incorrect PIN.

Boundary value analysis:

Test the minimum allowed amount and values just below/above it; the maximum single withdrawal and values just below/above it; the daily limit and values around it; and balances just below, equal to, and just above the requested amount.

Likely technique and adequacy:

The observed input-output behaviour suggests black-box testing, particularly equivalence partitioning and boundary value analysis. If tests only cover normal withdrawals and one or two errors, they are inadequate because they miss limits, invalid formats, blocked cards, insufficient ATM cash, and repeated failures.

Improved test cases:

1. Valid card/PIN, valid amount, sufficient balance – cash is dispensed and balance is reduced correctly.

2. Amount is zero – reject with a clear validation message.

3. Negative amount – reject.

4. Amount below the minimum – reject.

5. Amount at the minimum – accept if all other conditions are valid.

6. Amount at the maximum limit – accept if permitted.

7. Amount above the single-transaction limit – reject.

8. Amount exceeds daily limit – reject.

9. Balance is less than amount – reject without changing balance.

10. Balance equals amount – accept if overdraft and account rules permit.

11. Invalid PIN – reject and update failed-attempt count.

12. Blocked/expired card – reject.

13. ATM cash is insufficient – do not debit the account.

14. Network timeout during processing – ensure transaction status is reconciled and no duplicate debit occurs.

15. Repeat the same request – verify duplicate withdrawal is prevented.

Conclusion:

A strong reconstructed strategy combines equivalence partitioning, boundary value analysis, negative testing, and transaction-integrity checks. Actual limits and exact error messages must be confirmed from the system specification or further observations.

2. Reverse engineer a white-box testing approach from execution traces and code snippets.

Answer

Introduction:
White-box testing uses knowledge of internal code, decisions, loops, and execution paths. Reverse engineering the approach means rebuilding the control flow graph (CFG) and determining which paths the existing tests execute. The question does not include actual code or traces, so the following is a conceptual example.

Conceptual logic:

Start → read input → [input valid?]

- No → display error → End
- Yes → [amount greater than zero?]
  - No → display amount error → End
  - Yes → process transaction → [retry required?]
    - Yes → retry step (loop)
    - No → display result → End

Decision points:

1. Input validity.

2. Amount validity.

3. Retry condition.

Independent paths:

P1: Start → invalid input → error → End.

P2: Start → valid input → invalid amount → error → End.

P3: Start → valid input → valid amount → process → no retry → result → End.

P4: Start → valid input → valid amount → process → retry → process again → no retry → result → End.

Coverage evaluation:

Statement coverage checks whether every executable statement runs at least once. Branch coverage checks both true and false outcomes of every decision. Path coverage checks execution paths, although loops can create infinitely many paths, so practical testing limits loop iterations and tests zero, one, and multiple repetitions.

Reconstructed strategy:

1. Create a CFG from the actual source code.

2. List every decision and loop.

3. Design at least one test for each branch outcome.

4. Test loop entry, loop exit, zero iterations where possible, one iteration, and repeated iterations.

5. Add tests for exceptions and error-handling paths.

6. Record executed statements, branches, and paths using a coverage tool.

7. Review uncovered paths and add tests.

Conclusion:

A complete strategy should be based on the real code and traces. Because no specific code snippets or traces were included in the task sheet, this CFG is illustrative; exact paths and coverage percentages cannot be determined from the provided information.

3. Reverse engineer login and transaction testing from defect logs and results.

Answer

Introduction:
A web application login and transaction system contains authentication, session management, account validation, transaction entry, processing, and confirmation. Defect logs can suggest which testing techniques were used and where coverage is weak, but they cannot prove the original technique without test records.

Components:

1. Login input validation.

2. Username/password authentication.

3. Account lockout and session handling.

4. Transaction input and balance validation.

5. Transaction execution and database update.

6. Confirmation, audit log, and error handling.

Likely testing techniques:

• Equivalence partitioning: valid and invalid usernames/passwords, valid and invalid transaction amounts.

• Boundary value analysis: password length limits, login-attempt limit, transaction minimum/maximum, and available balance.

• White-box branch/path testing: successful login, failed login, lockout, transaction accepted, rejected, or rolled back.

• Security testing: session expiry, unauthorized access, injection attempts, and duplicate submissions.

Coverage gaps:

Defects in limits may indicate missing boundary tests. Defects in combined conditions may indicate insufficient branch or condition coverage. Duplicate transactions may indicate missing retry/idempotency tests. Session defects may indicate insufficient tests after logout or timeout.

Reconstructed test cases:

1. Correct username and password – login succeeds.

2. Incorrect password – login fails with a safe message.

3. Unknown username – access is denied.

4. Login attempts just below, at, and above the lockout threshold – verify policy.

5. Empty and overlong fields – handled safely.

6. Expired session accesses a protected page – user is redirected to login.

7. Valid transaction within balance and limits – transaction succeeds.

8. Amount at minimum and maximum boundaries – result follows specification.

9. Amount above limit or balance – rejected without account changes.

10. Network interruption during transaction – final status is consistent.

11. Same transaction request submitted twice – no duplicate debit.

12. Database failure – transaction is rolled back or marked pending safely.

13. Each decision outcome in transaction code – exercised by white-box tests.

14. Unauthorized user attempts another account's transaction – denied.

Integrated strategy:

Use equivalence partitioning and boundary value analysis for input combinations, then use CFG-based branch/path testing for internal decisions. Add security, recovery, and database consistency tests. Map every defect to a test case and rerun regression tests after fixes.

Conclusion:

Combining black-box and white-box methods offers better coverage than relying only on successful login and transaction scenarios.

4. Reverse engineer stock updates and low-stock alerts from inventory logs.

Answer

Introduction:
The inventory system updates stock when goods are received, sold, returned, or adjusted. Logs of successful and failed transactions can reveal rules such as minimum stock, permitted quantities, and the low-stock threshold.

Functional units:

1. Read product ID and quantity.

2. Verify that the product exists and is active.

3. Validate the quantity and transaction type.

4. Calculate the new stock value.

5. Save the update to the database.

6. Compare new stock with the low-stock threshold.

7. Create or send an alert.

8. Record the transaction in an audit log.

Equivalence partitions:

• Product: existing ID / unknown ID / inactive product.

• Quantity: positive valid quantity / zero / negative / invalid format.

• Stock result: above threshold / equal to threshold / below threshold / negative stock.

• Transaction type: receipt / sale / return / adjustment.

• Alert state: alert required / alert not required / alert already active.

Boundary values:

Test stock immediately below, exactly at, and immediately above the alert threshold. Test sale quantities that leave stock at zero, one unit, or below zero. Test the maximum permitted update quantity and values just outside it. The actual threshold and maximum must be taken from the system rules.

Reconstructed black-box tests:

1. Add valid quantity to an existing product – stock increases correctly.

2. Sell quantity available in stock – stock decreases correctly.

3. Sell exactly the available quantity – stock becomes zero if allowed.

4. Sell one unit more than available – reject or follow the documented back-order rule.

5. Enter zero or negative quantity – reject unless a defined adjustment operation allows it.

6. Use unknown product ID – show an error and make no change.

7. New stock just above threshold – no low-stock alert.

8. New stock equal to threshold – verify the specified alert rule.

9. New stock below threshold – alert is created.

10. Repeated update request – ensure stock is not changed twice.

11. Database failure during update – avoid partial updates.

12. Concurrent sales – prevent lost updates and negative stock.

Justification:

These cases cover valid and invalid input classes, boundaries, alert conditions, and transaction integrity. Logs should be compared against expected results to find missing tests. Each test should record initial stock, operation, expected final stock, and expected alert.

Conclusion:

The reconstructed suite goes beyond observed successful and failed examples and targets boundary errors, duplicate processing, concurrency, and alert reliability.

5. Reverse engineer white-box testing requirements from an executable and CPU/memory profile.

Answer

Introduction:
A compiled executable does not expose its source code directly. CPU and memory profiles show where resources are used, but spikes alone cannot identify an exact loop or decision. They can guide investigation and black-box stress testing.

Possible components:
1. Input parsing and validation.
2. Library record search.
3. Borrow/return transaction processing.
4. Fine or due-date calculation.
5. Database/file access.
6. Report generation and memory cleanup.

Conceptual CFG:

Start → validate request → [valid?]

- No → error response → End
- Yes → search records → [record found?]
  - No → not-found response → End
  - Yes → process records in a loop → [more records?]
    - Yes → process next record
    - No → save result → [save successful?]
      - No → error/recovery → End
      - Yes → success response → End

Path-testing strategy:

1. Run a small valid request to establish baseline resource use.

2. Test invalid input and missing records.

3. Test one record, many records, and maximum supported records.

4. Exercise loop exit and repeated iterations.

5. Test successful and failed database writes.

6. Repeat requests under increasing load and concurrent users.

7. Monitor CPU, memory, response time, errors, and memory growth.

8. Run long-duration tests to detect leaks and resource exhaustion.

9. Verify that errors do not corrupt library data and that resources are released.

Coverage and stability:

With source code, construct an exact CFG and measure statement, branch, and path coverage. With only an executable and profile, the CFG is conceptual and the internal branches cannot be confirmed. Resource-intensive paths should be tested at realistic and extreme workloads.

Conclusion:

Profiling helps prioritize tests but is not a substitute for source-level coverage analysis. Stability improves when expensive paths, error recovery, and memory cleanup are tested under stress.

6. Reverse engineer a mobile payment gateway state machine.

Answer

Introduction:
A payment gateway may move through Pending, Success, Reversal, and Timeout states. State-transition testing checks whether every permitted transition works and forbidden transitions are rejected. The exact API JSON fields are not included in the task sheet, so this is a conceptual model.

States:

• Pending: transaction has started but is not final.

• Success: payment is confirmed.

• Reversal: a completed or attempted transaction is being reversed/refunded according to gateway rules.

• Timeout: no response was received within the permitted time.

Conceptual transition diagram:
Start → Pending
Pending → Success (valid confirmation received)
Pending → Reversal (failure/reversal instruction, if permitted)
Pending → Timeout (time limit expires)
Timeout → Success or Reversal only if the provider's reconciliation rules explicitly permit it
Success → Reversal only when a valid reversal process is supported
Final states must not return to Pending without a new transaction.

Test suite:

1. Create valid payment – state becomes Pending.

2. Pending receives valid success response – state becomes Success.

3. Pending reaches timeout – state becomes Timeout.

4. Pending receives valid reversal instruction – state changes as specified.

5. Invalid state value in request – reject with validation error.

6. Success receives duplicate success response – do not duplicate the debit.

7. Reversal request references unknown transaction – reject.

8. Reversal of an already reversed transaction – handle idempotently or reject per rules.

9. Late success response after timeout – reconcile according to documented policy.

10. Missing transaction ID or amount – reject.

11. Amount or currency mismatch – reject.

12. Unauthorized or malformed callback – reject and log.

13. Verify that every accepted transition is persisted and audited.

Coverage:

Create a transition table with current state, event, expected next state, and result. Include every legal transition and representative illegal transitions. Test timeout and duplicate messages because they often create inconsistent payment records.

Conclusion:

State-transition testing is the likely technique to reconstruct. Exact legal transitions must be confirmed from the API contract and the supplied JSON examples.

7. Reverse engineer an encrypted communication module using fuzzing and BVA.

Answer

Introduction:
The module accepts input parameters and returns encrypted output. Fuzzing supplies unexpected, malformed, or random inputs to discover crashes and validation errors. Boundary value analysis focuses on input sizes and values near allowed limits.

Functional components:

1. Input parser.

2. Length and format validation.

3. Authentication/key validation.

4. Encryption/decryption operation.

5. Output encoding.

6. Error handling and secure logging.

Test design:

Assume the specification defines minimum and maximum input lengths. Test lengths at minimum−1, minimum, minimum+1, maximum−1, maximum, and maximum+1. Also test empty input, very long input, invalid character encoding, missing fields, null values, malformed keys, and incorrect parameter types. Do not assume specific length limits without the specification.

Fuzzing cases:

1. Random byte sequences.

2. Very long strings and repeated characters.

3. Truncated input and incomplete messages.

4. Unexpected Unicode and binary data.

5. Missing or extra parameters.

6. Malformed key/nonce/initialization-vector values, where applicable.

7. Injection-like strings in fields that may later be logged or stored.

8. Repeated malformed requests to check stability and rate limiting.

Security justification:

Validate length before copying or processing buffers. Use safe memory-handling APIs and strict parsing. Reject malformed inputs consistently. Keep cryptographic keys out of logs and return generic errors that do not reveal secrets. Ensure input is treated as data, not executable commands. Run sanitizers and security analysis when source code is available.

Expected outcomes:

Valid inputs produce correctly formatted encrypted output. Invalid inputs produce controlled errors without crashes, memory corruption, data leakage, or partial processing. Record crash, response, and resource information for each fuzz case.

Conclusion:

BVA tests predictable limits, while fuzzing explores unexpected combinations. Together they help detect buffer overflow, parser errors, and injection risks, although testing alone cannot prove the module is completely secure.

8. Improve testing for an insurance premium calculator with 100% statement coverage.

Answer

Introduction:
Statement coverage only proves that each executable statement has run at least once. It does not prove that every decision outcome or combination of conditions has been tested. This explains how tests can report 100% statement coverage while still missing defects in multi-condition if statements.

Illustrative logic:

If (age is within allowed range) AND (coverage amount is valid) AND (risk category is acceptable), calculate premium; otherwise, show an error. The actual conditions are not supplied, so this is an example.

Conceptual CFG:

Start → evaluate condition A → [A true?]

• No → error → End

• Yes → evaluate condition B → [B true?]

- No → error → End
- Yes → evaluate condition C → [C true?]
  - No → error → End
  - Yes → calculate premium → display result → End

Independent paths:

P1: A false → error.

P2: A true, B false → error.

P3: A true, B true, C false → error.

P4: A true, B true, C true → calculate and display premium.

Reconstructed tests:

1. All conditions true – premium is calculated.

2. Condition A false while other inputs are valid – reject.

3. A true and B false – reject.

4. A and B true, C false – reject.

5. Test each numeric input below, at, and above its permitted boundary.

6. Test combinations where two or more conditions change together.

7. Verify the premium formula for different valid risk categories.

8. Verify rounding, minimum premium, maximum premium, and invalid data rules.

9. Confirm errors do not return a misleading premium.

Coverage evaluation:

Branch coverage requires true and false outcomes for each decision. Condition coverage requires each individual Boolean condition to be true and false. Modified Condition/Decision Coverage (MC/DC), when appropriate, checks that each condition can independently affect the overall decision. Full path coverage may be impractical if loops or many combinations exist, so define feasible paths and document limits.

Conclusion:

The improved suite checks decision outcomes and condition combinations, reducing the chance that a premium is calculated incorrectly. Exact paths must be updated to match the real source code.

9. Reverse engineer a cloud file synchronization algorithm from defect reports.

Answer

Introduction:
A cloud synchronization tool compares local and cloud files and transfers changes. Lost-file defects associated with file size, type, and network speed may indicate gaps in size partitioning, transfer interruption handling, conflict resolution, or retry logic.

Components:

1. File selection and metadata collection.

2. File-size and file-type validation.

3. Change detection using metadata or hashes.

4. Upload/download queue.

5. Network transfer and retry.

6. Conflict resolution.

7. Temporary-file and final-file handling.

8. Completion verification and logs.

Equivalence classes:

• Size: empty file, small file, normal file, maximum supported file, and oversized file.

• Type: supported type, unsupported type, extension mismatch, and unusual filename.

• Network: stable/fast, slow, intermittent, disconnected, and restored connection.

• Change state: unchanged file, modified file, deleted file, and simultaneous edits.

• Transfer result: complete, interrupted, corrupted, or duplicate transfer.

Revised test strategy:

1. Sync an empty file and verify its contents.

2. Sync a small and a large supported file.

3. Test maximum size and values just below/above the limit.

4. Attempt an unsupported or malformed file and verify safe handling.

5. Interrupt the network during upload and download.

6. Restore the connection and verify resume/retry without data loss.

7. Compare checksums before and after transfer.

8. Edit the same file on two devices and verify conflict handling.

9. Delete a file on one device and check the documented deletion policy.

10. Test many files simultaneously under stress.

11. Fill local storage or cloud quota and verify a clear failure.

12. Verify that temporary files are cleaned up and incomplete files are not shown as complete.

Why ECP, BVA, and stress testing:

ECP selects representative inputs from meaningful groups. BVA tests values at and around file-size and quota limits. Stress testing checks large numbers of files, concurrent devices, and slow networks. Recovery testing ensures interrupted transfers are retried safely.

Conclusion:

Checksum verification, atomic file replacement, reliable retry logic, and explicit conflict rules help prevent data loss. Defect reports should be mapped to regression tests after every fix.

10. Reverse engineer e-commerce search logic using decision-table testing.

Answer

Introduction:
An e-commerce search and filter module may combine keyword, category, price, brand, rating, stock, and sort criteria. SQL queries and result sets can reveal whether conditions are combined using AND, OR, and NOT. Without the actual queries, the following is a representative decision-table model.

Conditions:

C1: keyword matches.

C2: category matches.

C3: price is within the selected range.

C4: item is in stock.

Action: include product in the result set.

Example decision table:

Rule 1: Keyword yes, category yes, price yes, stock yes → include product.

Rule 2: Keyword yes, category no, price yes, stock yes → exclude if filters use AND.

Rule 3: Keyword yes, category yes, price no, stock yes → exclude.

Rule 4: Keyword yes, category yes, price yes, stock no → exclude when in-stock filter is selected.

Rule 5: Keyword no, category yes, price yes, stock yes → include only if keyword is optional/empty; otherwise exclude.

Rule 6: Category A OR category B selected → include products matching either category.

Rule 7: Exclusion filter NOT brand X → exclude brand X and include otherwise matching products from other brands.

Rule 8: No filters supplied → return the default product list according to the application specification.

Testing strategy:

1. Test each filter independently.

2. Test all filters together to verify AND behaviour.

3. Test alternative values in the same filter to verify OR behaviour.

4. Test excluded brands or categories to verify NOT behaviour.

5. Test empty search, no matching products, and special characters.

6. Test minimum and maximum price, and values just outside the boundaries.

7. Test pagination and sorting after filtering.

8. Compare SQL conditions with the returned products.

9. Check parameterized queries to prevent SQL injection.

10. Verify that result counts and pages remain consistent when filters change.

Completeness:

A decision table makes combinations visible and prevents important rules from being overlooked. If there are many conditions, combine equivalent rules or use pairwise testing, while ensuring critical combinations are still covered. Actual SQL and expected datasets are required to confirm the precise logic.

Conclusion:

The reconstructed model tests AND/OR/NOT logic, filter boundaries, empty results, pagination, and security. The table must be refined using the actual query examples and business rules.